

NanoVLLM++

A Comprehensive Technical Report on an Educational LLM Inference Engine

July 2026

Contents

1	Project Overview	3
1.1	Target Model	3
1.2	Hardware	3
1.3	Project Structure	4
2	Architecture & Implementation	4
2.1	config.py	4
2.1.1	EngineConfig	4
2.1.2	ModelConfig	5
2.2	sampling_params.py	5
2.3	engine/sequence.py	6
2.3.1	SequenceStatus	6
2.3.2	Sequence Fields	6
2.4	engine/block_manager.py	7
2.4.1	Block	7
2.4.2	compute_block_hash	7
2.4.3	BlockManager	7
2.5	engine/model_runner.py	8
2.5.1	SimpleSampler	8
2.5.2	KV Cache Allocation	9
2.5.3	run_prefill	9
2.5.4	run_decode	10
2.5.5	run()	10
2.6	engine/scheduler.py	11
2.6.1	ScheduledBatch	11
2.6.2	Scheduler	11
2.7	engine/llm_engine.py	12
2.7.1	Initialization Flow	12
2.7.2	_build_model_config	13
2.7.3	_estimate_kv_blocks	13
2.7.4	add_request	14
2.7.5	step()	14
2.7.6	generate()	15
2.8	llm.py	15
3	Scheduling System	15

3.1	Token Budget (token_budget.py)	15
3.2	MixedBatchScheduler (chunked_prefill.py)	16
3.3	ScheduledItem	16
3.4	Why Chunked Prefill Works	17
3.5	Token Budget vs Sequence Budget	17
4	Speculative Decoding	17
4.1	Proposer (proposer.py)	17
4.1.1	Proposer Interface	17
4.1.2	NGramProposer	17
4.1.3	GreedyVerifier	18
4.2	SpecDecodeManager (manager.py)	19
4.3	SpecMetrics	19
4.4	Why Speculative Decoding is Not Always Faster	20
5	Quantization	20
5.1	Weight-Only INT8 Quantization (int8_weight_only.py)	20
5.1.1	Per-Channel INT8 Quantization	20
5.1.2	Why Per-Channel > Per-Tensor	20
5.1.3	Memory Math	20
5.1.4	Int8WeightOnlyLinear	21
5.1.5	quantize_model_int8	21
5.2	Simulated INT4 Quantization	21
5.3	model_memory_report	21
6	Serving Layer	22
6.1	Overview	22
6.2	Request Models	22
6.3	Response Models	22
6.4	Streaming via SSE	23
6.5	start_server	23
6.6	metrics Endpoint	23
7	Multi-Layered Diagnostics	24
7.1	Layer 1: EngineProfiler	24
7.2	Layer 2: RequestTracker	24
7.3	Layer 3: PyTorch Profiler	25
7.4	Layer 3b: NVIDIA Nsight Systems	25
7.5	diagnose_slowdown()	26
7.6	How the Layers Stack	26
8	Benchmarking Methodology	27
8.1	Design Philosophy	27
8.2	bench_baseline.py	27
8.3	bench_chunked_prefill.py	27
8.4	bench_spec_decode.py	27
8.5	bench_quantization.py	28
8.6	bench_profiling.py	28
9	Benchmark Results	28
9.1	Baseline Comparison	28
9.2	Chunked Prefill	28
9.3	Speculative Decoding (OPT-125M)	29

9.4	Quantization (OPT-125M)	29
9.5	Profiling (Qwen3-0.6B)	29
10	GPU Memory Analysis	29
10.1	Model Weight Memory	29
10.2	KV Cache Memory Per Block	30
10.3	Total KV Blocks Estimation	30
10.4	Why <code>gpu_memory_utilization = 0.90</code>	30
11	Debugging Section	30
11.1	OPT vs Qwen3 Config Key Names	30
11.2	Initialization Order	31
11.3	KV Cache OOM on Qwen3	31
11.4	Gibberish Output Without KV Cache	31
11.5	Profiler Tick Placement	31
11.6	RequestTracker TPOT Calculation	31
12	Interview Preparation — Q&A	32
12.1	Core Inference Concepts	32
12.2	Speculative Decoding	33
12.3	Quantization	33
12.4	Serving	34
12.5	Diagnostics	34
13	Glossary	35
A	Codebase Lineage	37
B	Key Design Decisions	37
C	Future Work	37

1 Project Overview

NanoVLLM++ is a from-scratch educational implementation of a vLLM-style large language model (LLM) inference engine. It is designed to teach **every foundational concept** of production-grade inference serving through readable, self-contained Python code. The project name reflects its pedagogical lineage: “nano” for minimalism, “vLLM” for the architectural inspiration, and “++” for the extensions beyond core PagedAttention (chunked prefill, speculative decoding, quantization, profiling, and serving).

1.1 Target Model

The primary target model is **Qwen/Qwen3-0.6B**, a 604-million parameter transformer with the following architecture:

Attribute	Value
Parameters	604M
Layers (L)	28
Hidden size (d_{model})	1024
Attention heads (H)	16
KV heads (H_{kv})	8
Head dimension (d_h)	128
GQA ratio	2:1 ($H_{kv} = H/2$)
Vocabulary size	152,064
Precision	BF16

Table 1: Qwen3-0.6B architecture summary

A fallback model, **facebook/opt-125m**, is used for rapid iteration during development due to its smaller size (125M params, 12 layers, 12 heads, $d_h = 64$, no GQA).

1.2 Hardware

All development and benchmarking is performed on an **NVIDIA RTX 5050 Laptop GPU** with 8 GB of VRAM. This constrains the KV cache budget and makes memory management critical—a deliberate pedagogical choice. The Qwen3-0.6B model in BF16 consumes approximately 1.21 GB for weights, leaving roughly 6 GB for the KV cache after accounting for PyTorch runtime overhead.

1.3 Project Structure

Module	Purpose
config.py	EngineConfig and ModelConfig dataclasses
sampling_params.py	SamplingParams with validation
llm.py	Thin user-facing LLM facade
engine/	Core engine: sequences, blocks, model runner, scheduler
scheduling/	Token budget and chunked-prefill policies
spec_decode/	N-gram proposer, greedy verifier, manager
quantization/	INT8/INT4 weight-only quantization
server/	FastAPI OpenAI-compatible serving layer
observability/	Engine profiler, request tracker, trace events
benchmarks/	Self-contained benchmarking scripts

Table 2: Project module layout

2 Architecture & Implementation

This section provides a detailed walkthrough of every source file in the engine. Together, they implement a continuous batching inference pipeline that takes tokenized prompts and produces generated text.

2.1 config.py

2.1.1 EngineConfig

```

1 @dataclass
2 class EngineConfig:
3     block_size: int = 256
4     max_num_seqs: int = 256
5     max_num_batched_tokens: int = 2048
6     max_model_len: int = 8192
7     gpu_memory_utilization: float = 0.90
8     enforce_eager: bool = False
9     enable_prefix_caching: bool = True
10    enable_chunked_prefill: bool = False
11    max_prefill_chunk_size: int = 512
12    decode_priority: bool = True
13    reserve_full_prompt_blocks: bool = True
14    enable_spec_decode: bool = False
15    max_spec_tokens: int = 4
16    spec_proposer_type: str = "ngram"
17    reserve_lookahead_slots: bool = True

```

Listing 1: EngineConfig dataclass

Each parameter teaches a specific runtime concept:

- **block_size** (256): The atomic unit of KV cache paging. Larger blocks reduce fragmentation but waste memory for short sequences. Smaller blocks improve utilization at the cost of more lookups. 256 is the same default used in vLLM.
- **max_num_batched_tokens** (2048): The total token budget per engine step. This is the key fairness knob—it limits the total number of tokens (prefill + decode) processed in one forward pass, preventing long prompts from starving short ones.

- **gpu_memory_utilization** (0.90): What fraction of total GPU memory the engine is allowed to use. The remaining 10% is headroom for PyTorch allocator overhead, CUDA context, and temporary tensors.
- **enforce_eager** (False): When **False**, the runtime may use CUDA graphs for decoding to amortize kernel launch overhead. Set to **True** for profiling/debugging.
- **enable_chunked_prefill** (False): When **True**, long prompt prefill is split into **max_prefill_chunk_size** chunks, and decode steps are interleaved between chunks.
- **decode_priority** (True): In mixed-batch scheduling, allocate token budget to decode items first.
- **reserve_lookahead_slots** (True): Reserve extra block slots for speculative tokens.

2.1.2 ModelConfig

```

1 @dataclass
2 class ModelConfig:
3     model_path: str = ""
4     num_layers: int = 0
5     num_kv_heads: int = 0
6     num_attention_heads: int = 0
7     head_dim: int = 0
8     hidden_size: int = 0
9     intermediate_size: int = 0
10    vocab_size: int = 0
11    dtype: str = "auto"
12    tie_word_embeddings: bool = False
13    rope_theta: float = 10000.0

```

Listing 2: ModelConfig dataclass

ModelConfig is extracted from the HuggingFace model config dictionary. The extraction logic handles cross-model key differences (e.g., **intermediate_size** in Qwen3/Llama vs **ffn_dim** in OPT).

2.2 sampling_params.py

```

1 @dataclass
2 class SamplingParams:
3     temperature: float = 1.0
4     max_tokens: int = 64
5     ignore_eos: bool = False
6     top_k: int = 0
7     top_p: float = 1.0
8     min_p: float = 0.0
9     stop: str | list[str] | None = None
10    stop_token_ids: list[int] | None = None
11    seed: int | None = None
12    logprobs: int | None = None
13    prompt_logprobs: bool = False

```

Listing 3: SamplingParams dataclass

All parameters are validated in `__post_init__`: temperature must be ≥ 0 , **max_tokens** must be positive, **top_p** must be in $[0, 1]$, and string **stop** values are normalized to lists.

Key parameters:

- **temperature = 0**: Greedy decoding (argmax). GPT-4 and production systems use this by default.
- **temperature > 0**: Scales logits before softmax. Higher values make output more random.
- **top_k**: Only sample from the top- k most likely tokens.
- **top_p** (nucleus sampling): Sample from the smallest set of tokens whose cumulative probability exceeds p .
- **logprobs**: If set, return log probabilities for the top- N tokens at each position.

2.3 engine/sequence.py

The `Sequence` class is the runtime representation of a generation request. It carries all state needed by the scheduler, block manager, and model runner.

2.3.1 SequenceStatus

```

1 class SequenceStatus:
2     WAITING = "WAITING"
3     RUNNING = "RUNNING"
4     FINISHED = "FINISHED"
5     PREEMPTED = "PREEMPTED"
```

Listing 4: SequenceStatus enum

State machine:

WAITING → RUNNING → FINISHED

WAITING → RUNNING → PREEMPTED → WAITING

Preemption (returning a running sequence to the waiting queue) occurs when the KV cache is full and a higher-priority request arrives. In the educational implementation, preemption is defined but not yet triggered.

2.3.2 Sequence Fields

```

1 class Sequence:
2     counter: int = 0
3
4     def __init__(self, token_ids, sampling_params, request_id=None):
5         self.seq_id = Sequence.counter # auto-increment
6         self.request_id = request_id or f"req-{self.seq_id}"
7         self.status = SequenceStatus.WAITING
8         self.sampling_params = sampling_params
9
10        self.token_ids = list(token_ids) # all tokens
11        self.prompt_len = len(token_ids) # immutable
12        self.num_computed_tokens = 0 # how many prefill
13        tokens done
14        self.num_generated_tokens = 0 # how many decode
15        tokens done
16        self.num_cached_tokens = 0 # prefix cache hits
17
18        self.block_table = [] # logical $ \
19        rightarrow$ physical block IDs
20        self.seq_len_at_capture = 0 # for CUDA graph
21        snapshots
```

```

19         self.enqueue_time = None                # wall time: added
    to scheduler
20         self.first_token_time = None            # wall time: first
    token sampled
21         self.completion_time = None            # wall time:
    sequence finished

```

Listing 5: Sequence class (fields)

Key properties:

- `last_token`: Returns `token_ids[-1]`, used as the single input for decode steps.
- `uncomputed_tokens`: `prompt_len - num_computed_tokens`. In full prefill, this is the whole prompt. In chunked prefill, it decreases each chunk.
- `append_token(tid)`: Appends a token and increments `num_generated_tokens`.

2.4 engine/block_manager.py

This is the centerpiece of PagedAttention. It manages KV cache memory through a logical-to-physical block mapping.

2.4.1 Block

```

1 class Block:
2     def __init__(self, block_id: int):
3         self.block_id = block_id
4         self.ref_count = 0          # how many sequences share this block
5         self.hash = None            # chained hash for prefix cache lookup
6         self.token_ids = []         # tokens stored in this block

```

Listing 6: Block class

Each **Block** represents a physical unit of KV cache memory. With `block_size=256`, each block stores KV tensors for 256 token positions across all layers and heads. The `ref_count` enables safe sharing: two sequences that share a prefix can reference the same physical block, and the block is freed only when `ref_count` drops to zero.

2.4.2 compute_block_hash

```

1 def compute_block_hash(parent_hash, block_tokens):
2     return hash((parent_hash, tuple(block_tokens)))

```

Listing 7: Chained block hashing

This is **chained hashing**: each block’s hash depends on the parent block’s hash and its own tokens. This means identical token prefixes across different sequences produce identical hashes, enabling zero-overhead prefix cache reuse. Python’s built-in `hash()` on tuples is sufficient for a single-process educational implementation; production systems use xxHash or SHA-256 for cross-instance consistency.

2.4.3 BlockManager

```

1 class BlockManager:
2     def __init__(self, num_blocks: int, block_size: int):
3         self.blocks = [Block(i) for i in range(num_blocks)]
4         self.free_block_ids = set(range(num_blocks))

```

```

5     self.used_block_ids = set()
6     self.hash_to_block_id: dict = {}

```

Listing 8: BlockManager (key methods)

Core operations:

1. **required_blocks(num_tokens)**: $\lceil \frac{\text{num_tokens}}{\text{block_size}} \rceil$. The ceiling division is critical—a sequence with 257 tokens needs 2 blocks, not 1.
2. **allocate(seq)**: Allocates blocks for an entire prompt. For each full block:
 - Computes the chained hash.
 - Checks `hash_to_block_id` for a cache hit.
 - If found, increments `ref_count` and increments `seq.num_cached_tokens`—the prefix cache hit counter.
 - If not found, allocates a fresh block and registers the hash.
 The final partial block (if `prompt_len` is not a multiple of `block_size`) is always freshly allocated.
3. **allocate_fresh_block()**: Pops from `free_block_ids`, pushes to `used_block_ids`, sets `ref_count = 1`.
4. **may_append(seq)**: Checks if the current logical block is full. If so, allocates a new physical block and appends it to `seq.block_table`. Returns `True` if allocation succeeded.
5. **deallocate(seq)**: Iterates over `seq.block_table`, decrements each block’s `ref_count`. When `ref_count` hits 0, the block is returned to `free_block_ids`. Note: the hash remains in `hash_to_block_id` so future sequences can still find it.
6. **hash_prompt_blocks(token_ids)**: Computes chained hashes for all full blocks in a token list. Used for pre-checking which blocks are already cached before allocating.

Teaching points:

- **Logical-to-physical mapping**: `seq.block_table[i]` maps logical block i to a physical block ID. This is the core of PagedAttention—it decouples the sequence’s view of memory from actual allocation.
- **Reference counting**: Prevents use-after-free when two sequences share a block; the block is freed only when the last reference drops.
- **Copy-on-write**: When a cached block is found and its `ref_count` was 0 (freed but hash still registered), it is re-used. When `ref_count > 0`, the block is shared—if one sequence later modifies its KV, a copy must be made.
- **Prefix caching**: If two prompts share the first 512 tokens, they each get the same first two block hashes, and the BlockManager reuses those physical blocks.

2.5 engine/model_runner.py

ModelRunner is the interface between the scheduler and the actual HuggingFace model. It handles prefill and decode execution separately.

2.5.1 SimpleSampler

```

1 class SimpleSampler:
2     def apply_temperature(self, logits, temperature):
3         if temperature == 0: return logits
4         return logits / temperature
5
6     def apply_top_k(self, logits, top_k):
7         if top_k <= 0: return logits

```

```

8         threshold = logits.topk(top_k, dim=-1).values[... , -1, None]
9         logits[logits < threshold] = float("-inf")
10        return logits
11
12    def apply_top_p(self, logits, top_p):
13        sorted_logits, sorted_indices = logits.sort(dim=-1, descending=True)
14        cumulative_probs = sorted_logits.softmax(dim=-1).cumsum(dim=-1)
15        remove_mask = cumulative_probs > top_p
16        remove_mask[... , 1:] = remove_mask[... , :-1].clone()
17        remove_mask[... , 0] = False
18        indices_to_remove = remove_mask.scatter(-1, sorted_indices,
remove_mask)
19        logits[indices_to_remove] = float("-inf")
20        return logits

```

Listing 9: SimpleSampler: temperature

The sampler supports three sampling strategies, applied in order:

1. Temperature scaling
2. Top- k filtering (keep only the k most probable tokens)
3. Top- p / nucleus sampling (keep tokens until cumulative probability $> p$)

When `temperature == 0`, it returns `argmax` (greedy). Otherwise, it samples from the resulting probability distribution using `torch.multinomial`.

2.5.2 KV Cache Allocation

```

1 def _allocate_kv_cache(self):
2     self.kv_keys = None
3     self.kv_values = None

```

Listing 10: Simplified KV cache (uses HF `past_key_values`)

In the current implementation, the engine does **not** maintain its own pre-allocated paged KV tensors. Instead, each `Sequence` object stores the HuggingFace `past_key_values` tuple on the `_kv_cache` attribute. This was a deliberate simplification after discovering that a manually pre-allocated paged KV cache consumed 2.57GB for Qwen3-0.6B on the 8 GB RTX 5050, leaving insufficient room for model weights and runtime overhead. The `past_key_values` approach lets HF manage memory internally, which is adequate for an educational engine.

2.5.3 run_prefill

```

1 def run_prefill(self, seq, num_tokens):
2     start = seq.num_computed_tokens
3     end = min(start + num_tokens, len(seq.token_ids))
4     token_slice = seq.token_ids[start:end]
5     input_ids = torch.tensor(token_slice, dtype=torch.long).unsqueeze(0)
6
7     with torch.no_grad():
8         output = self.model(input_ids, use_cache=True)
9         last_logits = output.logits[:, -1, :]
10        past = output.past_key_values
11
12    seq._kv_cache = past
13
14    if end >= seq.prompt_len:
15        token_id, _ = self.sampler(last_logits[0], seq.sampling_params)
16        seq.append_token(token_id)
17        return token_id

```

```
18     return None
```

Listing 11: Prefill execution (per-sequence)

`run_prefill` processes `num_tokens` tokens from the prompt, starting at `num_computed_tokens`. For full prefill, `num_tokens` equals the entire prompt length. For chunked prefill, it processes one chunk at a time.

Key details:

- The token slice is `[start:end]`, where `start = num_computed_tokens` (how many prompt tokens have been processed so far) and `end` is capped at the total prompt length.
- `use_cache=True` tells HF to return `past_key_values`. This KV cache is stored on the sequence for subsequent decode steps.
- If the full prompt has been processed (`end >= prompt_len`), the model samples the first generated token and appends it. Otherwise, it returns `None` (chunked prefill in progress).

2.5.4 run_decode

```
1 def run_decode(self, seq):
2     past = getattr(seq, "_kv_cache", None)
3     input_ids = torch.tensor([[seq.last_token]], dtype=torch.long)
4
5     with torch.no_grad():
6         if past is not None:
7             output = self.model(input_ids, past_key_values=past, use_cache=
8             True)
9             seq._kv_cache = output.past_key_values
10        else:
11            output = self.model(input_ids, use_cache=False)
12            logits = output.logits[:, -1, :]
13
14        token_id, _ = self.sampler(logits[0], seq.sampling_params)
15        seq.append_token(token_id)
16    return token_id
```

Listing 12: Decode execution (per-sequence)

Decode processes **exactly one token per forward pass**—the last generated token. The KV cache from the previous step is passed as `past_key_values`, and the new KV cache is saved back to the sequence. This is the $O(1)$ per-step computation that makes decoding efficient: only the new token’s attention keys/queries/values are computed.

2.5.5 run()

```
1 def run(self, scheduled_batch):
2     results = []
3     for item in scheduled_batch.items:
4         seq = item.seq
5         if item.kind == "decode":
6             token_id = self.run_decode(seq)
7             results.append((seq, token_id))
8         elif item.kind == "prefill":
9             token_id = self.run_prefill(seq, item.num_tokens)
10            if token_id is not None:
11                results.append((seq, token_id))
12    return results
```

Listing 13: run() dispatcher

Iterates over all `ScheduledItems` in the batch and dispatches to `run_decode` or `run_prefill` based on `item.kind`. The results list contains `(seq, token_id)` tuples for completed work units.

2.6 engine/scheduler.py

The `Scheduler` manages the sequencing pipeline: admission from the waiting queue, block allocation, mixed-batch building, and postprocessing.

2.6.1 ScheduledBatch

```

1 class ScheduledBatch:
2     def __init__(self, items):
3         self.items = items    # list[ScheduledItem]
4
5     @property
6     def is_prefill(self):
7         return all(i.kind == "prefill" for i in self.items)
8
9     @property
10    def is_decode(self):
11        return all(i.kind == "decode" for i in self.items)

```

Listing 14: ScheduledBatch container

A `ScheduledBatch` is a simple container with convenience properties for checking batch composition. With chunked prefill enabled, a batch can contain both prefill and decode items simultaneously (a “mixed batch”).

2.6.2 Scheduler

```

1 class Scheduler:
2     def __init__(self, config, block_manager):
3         self.waiting = deque()
4         self.running = deque()
5         self.mixed_scheduler = MixedBatchScheduler(config, block_manager)

```

Listing 15: Scheduler class

The scheduler maintains two deques: `waiting` (sequences awaiting admission) and `running` (actively generating sequences). The `MixedBatchScheduler` handles the actual item selection when chunked prefill is enabled.

`schedule()` dispatch:

```

1 def schedule(self):
2     if self.config.enable_chunked_prefill:
3         items = self.mixed_scheduler.build_items(self.waiting, self.running)
4         return ScheduledBatch(items)
5
6     if self.waiting:
7         return self._schedule_prefill_only()
8     if self.running:
9         return self._schedule_decode_only()
10    return ScheduledBatch([])

```

Listing 16: `schedule()` logic

When chunked prefill is **disabled**, the scheduler uses a strict prefill-before-decode policy: it schedules as many waiting sequences as fit in the token budget, then (in subsequent steps) schedules decode-only batches. This is the behavior from the original vLLM.

When chunked prefill is **enabled**, the scheduler delegates to `MixedBatchScheduler.build_items()`, which interleaves prefill chunks and decode items.

postprocess():

```

1 def postprocess(self, scheduled_batch, output_tokens):
2     for item in scheduled_batch.items:
3         seq = item.seq
4         if item.kind == "prefill":
5             seq.num_computed_tokens += item.num_tokens
6             if seq.num_computed_tokens >= seq.prompt_len:
7                 seq.status = SequenceStatus.RUNNING
8                 self.running.append(seq)
9
10        elif item.kind == "decode":
11            if seq.num_generated_tokens >= seq.sampling_params.max_tokens:
12                seq.status = SequenceStatus.FINISHED
13                self.block_manager.deallocate(seq)
14                finished.append(seq)
15            elif not seq.sampling_params.ignore_eos and self._is_eos(seq):
16                seq.status = SequenceStatus.FINISHED
17                self.block_manager.deallocate(seq)
18                finished.append(seq)

```

Listing 17: Scheduler postprocess

After execution, postprocess transitions sequences through their lifecycle:

- **Prefill completion:** When `num_computed_tokens >= prompt_len`, the sequence moves from WAITING to RUNNING and joins the running deque.
- **Decode termination:** Checked in order—if `max_tokens` is reached, finish; if an EOS token is generated (and `ignore_eos` is false), finish. On finish, blocks are deallocated and the sequence is removed from the running deque.

2.7 engine/llm_engine.py

`LLMEngine` is the central orchestrator that wires together all components: configuration, tokenizer, model, block manager, model runner, and scheduler.

2.7.1 Initialization Flow

```

1 def __init__(self, model_path: str, **kwargs):
2     # 1. Build EngineConfig from kwargs
3     engine_config = EngineConfig()
4     for k, v in kwargs.items():
5         if hasattr(engine_config, k):
6             setattr(engine_config, k, v)
7
8     # 2. Load tokenizer
9     self.tokenizer = AutoTokenizer.from_pretrained(model_path)
10
11    # 3. Load model (BF16 on CUDA, FP32 on CPU)
12    self.model = AutoModelForCausalLM.from_pretrained(
13        model_path, torch_dtype=torch_dtype
14    ).to(self.device).eval()
15

```

```

16 # 4. Build ModelConfig from HF config
17 self.model_config = self._build_model_config(model_path)
18
19 # 5. Estimate KV blocks from available GPU memory
20 self.num_kv_blocks = self._estimate_kv_blocks()
21
22 # 6. Create BlockManager
23 self.block_manager = BlockManager(self.num_kv_blocks, engine_config.
24                                   block_size)
25
26 # 7. Create ModelRunner
27 self.model_runner = ModelRunner(self.model, self.model_config,
28                                  engine_config)
29
30 # 8. Create Scheduler
31 self.scheduler = Scheduler(engine_config, self.block_manager)

```

Listing 18: LLMEngine.*init_low*

This ordering is critical. The model must be loaded before `_estimate_kv_blocks()` runs, because estimating KV blocks requires knowing the model’s memory footprint. An earlier version had this reversed, causing an `AttributeError`.

2.7.2 `_build_model_config`

Extracts model dimensions from the HuggingFace config, handling cross-model key differences:

```

1 num_kv_heads = getattr(
2     hf_config, "num_key_value_heads",
3     getattr(hf_config, "num_attention_heads", 0)
4 )
5 intermediate_size = getattr(
6     hf_config, "intermediate_size",
7     getattr(hf_config, "ffn_dim", 0)
8 )
9 head_dim = getattr(
10     hf_config, "head_dim",
11     hf_config.hidden_size // hf_config.num_attention_heads
12 )

```

Listing 19: Robust config extraction

Qwen3 and Llama models use `num_key_value_heads` (for GQA), while OPT uses `num_attention_heads` for both. Qwen3 uses `intermediate_size`; OPT uses `ffn_dim`.

2.7.3 `_estimate_kv_blocks`

```

1 def _estimate_kv_blocks(self) -> int:
2     kv_per_token = 2 * num_layers * num_kv_heads * head_dim * dtype_bytes
3     kv_per_block = kv_per_token * block_size
4
5     total_gpu_memory = torch.cuda.get_device_properties(0).total_memory
6     gpu_available = total_gpu_memory * gpu_memory_utilization
7     model_memory = sum(p.numel() * p.element_size() for p in model.parameters())
8     available_for_kv = gpu_available - model_memory - 512 * 1024 * 1024
9
10    return max(10, int(available_for_kv // kv_per_block))

```

Listing 20: KV block estimation formula

The formula computes:

1. **KV per token:** $2 \times L \times H_{kv} \times d_h \times 2$ bytes. The factor of 2 is for keys + values. For Qwen3 (28 layers, 8 KV heads, $d_h = 128$): $2 \times 28 \times 8 \times 128 \times 2 = 114,688$ bytes per token.
2. **KV per block:** `kv_per_token` $\times$ 256.
3. **Available memory:** `gpu_total` $\times$ 0.90 $-$ `model_memory` $-$ 512MB.
4. **Number of blocks:** $\lfloor \text{available_for_kv} / \text{kv_per_block} \rfloor$, with a floor of 10 blocks.

2.7.4 add_request

```

1 def add_request(self, prompt: str, sampling_params: SamplingParams):
2     token_ids = self.tokenizer.encode(prompt, add_special_tokens=True)
3     seq = Sequence(token_ids, sampling_params)
4     seq.enqueue_time = time.time()
5     self.scheduler.add(seq)
6     if self._request_tracker:
7         self._request_tracker.record_enqueue(seq.request_id)
8     return seq.request_id

```

Listing 21: Request admission

Tokenizes the prompt, wraps it in a `Sequence`, records the enqueue timestamp, adds it to the scheduler, and returns a request ID.

2.7.5 step()

```

1 def step(self):
2     self.step_counter += 1
3     if self._profiler:
4         self._profiler.start_step(self.step_counter)
5         self._profiler.tick("schedule")
6
7     scheduled = self.scheduler.schedule()
8     if self._profiler:
9         self._profiler.tick("schedule")
10
11     if not scheduled:
12         return []
13
14     model_kind = "prefill" if getattr(scheduled, "is_prefill", True) else "
decode"
15     if self._profiler:
16         self._profiler.tick(model_kind)
17
18     output_tokens = self.model_runner.run(scheduled)
19
20     if self._profiler:
21         self._profiler.tick(model_kind)
22         self._profiler.tick("postprocess")
23
24     finished = self.scheduler.postprocess(scheduled, output_tokens)
25
26     if self._profiler:
27         self._profiler.tick("postprocess")
28         self._profiler.end_step(...)
29
30     return finished

```

Listing 22: Engine step with profiler instrumentation

Each engine step performs **schedule** → **model forward** → **postprocess**, with optional profiler ticks. The profiler ticks measure the wall time between consecutive `tick()` calls:

- `tick("schedule")` (twice): First call starts measurement; second call records elapsed time as `schedule_ms`.
- `tick("prefill")` / `tick("decode")`: Wraps the `model_runner.run()` call. The elapsed time between the two ticks is the model forward time.
- `tick("postprocess")`: Wraps the `scheduler.postprocess()` call.

2.7.6 generate()

The `generate` loop iterates `step()` while `scheduler.has_unfinished()` returns `True`, collecting finished sequences and post-processing them into output dictionaries with decoded text, generated text, token IDs, and token counts.

2.8 llm.py

```
1 class LLM(LLMEngine):
2     pass
```

Listing 23: LLM: thin facade over LLMEngine

LLM is a zero-logic subclass of `LLMEngine`. This separation serves three purposes:

1. Users interact only with LLM, never with engine internals.
2. The engine can be refactored without changing the public API.
3. Tests can exercise the engine directly without importing the LLM wrapper.

3 Scheduling System

The scheduling system is the brain of the engine. It decides **which** sequences to process and **how many** tokens to process in each engine step.

3.1 Token Budget (token_budget.py)

```
1 class TokenBudget:
2     def __init__(self, max_tokens, max_seqs=256, decode_priority=True,
3                 max_prefill_chunk=512):
4         self.max_tokens = max_tokens
5         self.remaining = max_tokens
6         self.scheduled_count = 0
7         self.decode_priority = decode_priority
8         self.max_prefill_chunk = max_prefill_chunk
9
10    def can_fit(self, num_tokens: int) -> bool:
11        return num_tokens <= self.remaining and \
12               self.scheduled_count < self.max_seqs
13
14    def consume(self, num_tokens: int):
15        self.remaining -= num_tokens
16        self.scheduled_count += 1
17
18    def prefill_chunk_size(self, remaining_prompt: int) -> int:
19        return min(remaining_prompt, self.remaining, self.max_prefill_chunk)
```

Listing 24: TokenBudget class

The token budget is the **primary fairness mechanism** in vLLM-style schedulers. Instead of limiting the *number of sequences* per step, it limits the *number of tokens*. This prevents a single 4096-token prompt from monopolizing the GPU while dozens of decode-only sequences wait.

Instance lifecycle: A new `TokenBudget` is created for each scheduler invocation, consuming tokens as items are added. When **remaining** reaches zero, scheduling stops for this step. At the next engine step, a fresh budget is created.

3.2 MixedBatchScheduler (chunked_prefill.py)

```

1 def build_items(self, waiting, running):
2     budget = TokenBudget(max_tokens=max_num_batched_tokens, ...)
3     items = []
4
5     if budget.decode_priority:
6         self._schedule_decode(running, budget, items) # Phase A
7         self._schedule_prefill_chunks(waiting, budget, items) # Phase B
8     if not budget.decode_priority:
9         self._schedule_decode(running, budget, items) # Phase A (alternate)
10
11     return items

```

Listing 25: `MixedBatchScheduler.build_items` algorithm

Two-phase algorithm:

1. **Phase A — Decode** (`_schedule_decode`): Iterates over the `running` deque. For each sequence, checks if a block can be appended (`can_append`), reserves the block (`may_append`), and creates a `ScheduledItem(kind="decode", num_tokens=1)`. Each decode item consumes exactly 1 token from the budget.
2. **Phase B — Prefill Chunks** (`_schedule_prefill_chunks`): Iterates over the `waiting` deque. For each sequence:
 - If `num_computed_tokens == 0` (first admission): attempts full block allocation.
 - Computes `chunk = budget.prefill_chunk_size(remaining)`.
 - Creates a `ScheduledItem(kind="prefill", num_tokens=chunk)`.
 - If `chunk >= remaining` (all prompt tokens covered), pops the sequence from `waiting`. Otherwise, leaves it in `waiting` with accumulated `num_computed_tokens`.

When `decode_priority` is `True` (default), Phase A runs before Phase B. This ensures that already-running sequences continue generating tokens smoothly, keeping TPOT (time per output token) low, while new prefill work uses whatever budget remains.

3.3 ScheduledItem

```

1 @dataclass
2 class ScheduledItem:
3     seq: object
4     kind: str # "prefill" or "decode"
5     num_tokens: int # e.g., 512 for a prefill chunk, 1 for decode

```

Listing 26: `ScheduledItem` dataclass

A `ScheduledItem` is the atomic scheduling unit. It represents a single work item for the model runner: either process N prefill tokens or generate 1 decode token.

3.4 Why Chunked Prefill Works

Consider a workload with one 2048-token prompt and 5 short-running decode sequences:

	No Chunking	Chunked (512-token chunks)
Step 1	Prefill 2048 tokens for seq A	Decode 5 seqs + Prefill 512 tokens for seq A
Step 2	Decode 5 seqs	Decode 5 seqs + Prefill 512 tokens for seq A
Step 3	Decode 5 seqs	Decode 5 seqs + Prefill 512 tokens for seq A
Step 4	Decode 5 seqs	Decode 5 seqs + Prefill 512 tokens for seq A

Without chunking, the 5 decoding sequences wait for the entire 2048-token prefill to complete. With chunking, decode steps interleave between prefill chunks. **TTFT for short requests drops significantly** because they don’t wait for the long prompt to finish.

3.5 Token Budget vs Sequence Budget

A naive scheduler might limit batches to `max_num_seqs`. This falls apart with heterogeneous sequence lengths:

- If `max_num_seqs = 32` and 32 sequences each have 4096-token prompts, the scheduler must process 131,072 tokens in one step—likely exceeding GPU memory and causing OOM.
- If 1 sequence has a 4096-token prompt and 127 sequences are in decode (1 token each), a sequence-count limit can’t express “let the 127 decoders run while chunking the long prefill.”

Token budget solves both problems: it caps the total computation per step, regardless of how many sequences that computation is distributed across.

4 Speculative Decoding

Speculative decoding reduces latency by generating multiple tokens per target-model forward pass. The key insight is that **decoding is memory-bandwidth-bound**: reading model weights from GPU memory dominates runtime, so processing k tokens in one forward pass costs roughly the same as processing 1 token (amortized weight reads).

4.1 Proposer (proposer.py)

4.1.1 Proposer Interface

```

1 class Proposer:
2     def propose(self, seq, max_tokens: int) -> Proposal:
3         raise NotImplementedError

```

Listing 27: Proposer abstract interface

The proposer generates a sequence of candidate tokens that the target model will verify. The interface is deliberately abstract to allow plugging in different proposal strategies (n-gram, draft model, Medusa heads, etc.).

4.1.2 NGramProposer

```

1 class NGramProposer(Proposer):
2     def __init__(self, min_ngram=2, max_ngram=5):
3         self.min_ngram = min_ngram
4         self.max_ngram = max_ngram
5
6     def propose(self, seq, max_tokens):

```

```

7     tokens = seq.token_ids
8     for n in range(min(max_ngram, len(tokens)-1), min_ngram-1, -1):
9         suffix = tokens[-n:]
10        for i in range(len(tokens) - n - 1, -1, -1):
11            if tokens[i:i+n] == suffix:
12                start = i + n
13                proposal = tokens[start:start + max_tokens]
14                if proposal:
15                    return Proposal(list(proposal))
16    return Proposal([])

```

Listing 28: NGramProposer algorithm

Algorithm:

1. Start with the longest n-gram (5 tokens). Take the last n tokens as the *suffix*.
2. Search backward through the sequence for another occurrence of this suffix.
3. If found, the tokens immediately following that occurrence are the proposal.
4. If no match at $n = 5$, try $n = 4$, then $n = 3$, then $n = 2$.
5. If no match at any n-gram size, return an empty proposal.

Example: For the sequence “The cat sat on the mat. The cat”, the suffix “The cat” (last 2 tokens) matches at position 0. The tokens that followed at position 0 are “sat on the mat.”—this becomes the proposal.

Teaching point: N-gram proposals require **no second model** and **no extra GPU memory**. They are purely algorithmic. This makes them an excellent entry point for understanding speculative decoding without the complexity of draft models or Medusa heads. For repetitive text (structured data, code, poetry), n-gram acceptance rates can be surprisingly high.

4.1.3 GreedyVerifier

```

1 class GreedyVerifier:
2     def verify(self, target_logits, proposal):
3         accepted = []
4         for i, proposed in enumerate(proposal.token_ids):
5             target_token = int(torch.argmax(target_logits[i]))
6             if proposed == target_token:
7                 accepted.append(proposed)
8             else:
9                 accepted.append(target_token)
10                rejection_token = target_token
11                break
12    return accepted, rejection_token

```

Listing 29: GreedyVerifier verify algorithm

Algorithm:

1. For each proposed token:
 - Get the target model’s greedy prediction (**argmax**) at that position.
 - If it matches the proposed token, accept it.
 - If it doesn’t match, accept the target model’s token instead and **stop**—remaining proposals are discarded.
2. Returns: (**accepted_tokens**, **rejection_token**).

Important: This is **greedy verification**, not the full speculative sampling algorithm from the Leviathan et al. (2023) paper. Greedy verification guarantees correct greedy decoding but does **not** guarantee identical output distribution to non-speculative decoding under temperature

> 0 . For that, rejection sampling with probability matching is required. The educational version deliberately uses the simpler greedy verifier.

4.2 SpecDecodeManager (manager.py)

```

1 def step(self, seq) -> int:
2     proposal = self.proposer.propose(seq, self.max_spec_tokens)
3     self.metrics.proposals_made += 1
4
5     if not proposal.token_ids:
6         self.metrics.proposals_empty += 1
7         token_id = self.model_runner.run_decode(seq)
8         self.metrics.target_forwards += 1
9         return 1
10
11     self.metrics.tokens_proposed += len(proposal.token_ids)
12
13     # Run target model on proposed tokens
14     ids = torch.tensor(proposal.token_ids).unsqueeze(0)
15     with torch.no_grad():
16         output = self.model_runner.model(ids, use_cache=False)
17         logits = output.logits[0]
18
19     self.metrics.target_forwards += 1
20
21     accepted, _ = self.verifier.verify(logits, proposal)
22     self.metrics.tokens_accepted += len(accepted)
23
24     for tid in accepted:
25         seq.append_token(tid)
26
27     return len(accepted)

```

Listing 30: SpecDecodeManager.step() flow

Full flow:

1. **Propose:** The n-gram proposer scans for a matching suffix and returns up to `max_spec_tokens` candidates.
2. **Fallback:** If the proposer returns empty, fall back to standard 1-token decode.
3. **Target forward:** Feed the proposed tokens to the target model in one batch. The model runs `use_cache=False` because there’s no KV cache for speculative tokens—this is a one-shot forward.
4. **Verify:** Compare each model prediction against the proposal.
5. **Accept:** Append accepted tokens to the sequence. The number of accepted tokens is returned as the step’s “tokens generated” count.

4.3 SpecMetrics

```

1 @dataclass
2 class SpecMetrics:
3     proposals_made: int = 0
4     proposals_empty: int = 0
5     tokens_proposed: int = 0
6     tokens_accepted: int = 0
7     target_forwards: int = 0
8
9     @property

```

```

10 def acceptance_rate(self):
11     return self.tokens_accepted / self.tokens_proposed

```

Listing 31: SpecMetrics tracking

Key metric: $\text{acceptance_rate} = \frac{\text{tokens_accepted}}{\text{tokens_proposed}}$. An acceptance rate of 0.75 means 3 out of every 4 proposed tokens are accepted, giving a theoretical 4 $\times$ speedup (4 tokens per model forward vs 1). Real speedup is lower due to proposer overhead and verification cost.

4.4 Why Speculative Decoding is Not Always Faster

At high QPS (queries per second), the GPU is already saturated with decode batches. Adding speculative overhead (proposal computation, extra verification forward) can actually *slow down* throughput because the proposer competes for compute. Speculation is most effective at **low QPS** where decode is memory-bandwidth-bound and there’s compute headroom. This is the classic “latency vs throughput” tradeoff.

5 Quantization

5.1 Weight-Only INT8 Quantization (int8_weight_only.py)

5.1.1 Per-Channel INT8 Quantization

```

1 def quantize_per_channel_int8(weight: torch.Tensor):
2     max_abs = weight.abs().amax(dim=1, keepdim=True).clamp(min=1e-8)
3     scale = max_abs / 127.0
4     qweight = torch.round(weight / scale).clamp(-128, 127).to(torch.int8)
5     return qweight, scale.squeeze(1)

```

Listing 32: quantize_per_channel_int8

Each row of the weight matrix gets its own scale factor. **Per-channel** scaling preserves more accuracy than **per-tensor** scaling because different output channels can have dramatically different magnitude ranges—a channel corresponding to a common token like “the” may have much larger weights than one for a rare token.

5.1.2 Why Per-Channel > Per-Tensor

Consider a weight matrix $W \in \mathbb{R}^{4096 \times 4096}$. Per-tensor quantization computes one scale $\alpha = \max(|W|)/127$ for the entire matrix. If row 5 has values in $[-0.001, 0.001]$ but row 42 has values in $[-1.0, 1.0]$, per-tensor scaling maps row 5’s values to $[-0.1, 0.1]$ in the quantized domain—effectively using only 10% of the quantization range and losing precision. Per-channel gives each row its own scale, using the full $[-128, 127]$ range for every row.

5.1.3 Memory Math

Format	Bytes/Weight	604M Model	vs FP16
FP32	4	2.42 GB	2 $\times$
FP16	2	1.21 GB	1 $\times$ (baseline)
INT8	1	0.60 GB	0.5 $\times$
INT4	0.5	0.30 GB	0.25 $\times$

5.1.4 Int8WeightOnlyLinear

```

1 class Int8WeightOnlyLinear(torch.nn.Module):
2     def forward(self, x):
3         weight = dequantize_int8(self.qweight, self.scale).to(x.dtype)
4         out = x @ weight.T
5         if self.bias is not None:
6             out = out + self.bias
7         return out

```

Listing 33: Int8WeightOnlyLinear forward pass

This layer stores weights as `int8` buffers but dequantizes to FP32/FP16 on every forward pass. **Naive dequantization saves memory but may not improve speed:** the dequantization step adds overhead, and the matrix multiplication still runs in FP16/FP32. Production systems use **fused dequantize + matmul kernels** (implemented in Triton or CUDA, e.g., AWQ, GPTQ) that fuse the dequantization into the matrix multiply, reading INT8 weights directly and performing the multiply-accumulate in higher precision.

5.1.5 quantize_model_int8

```

1 def quantize_model_int8(model, exclude_keywords=("lm_head", "embed")):
2     for name, module in list(model.named_modules()):
3         if not isinstance(module, torch.nn.Linear):
4             continue
5         if any(kw in name for kw in exclude_keywords):
6             skipped_count += 1
7             continue
8         # Replace with Int8WeightOnlyLinear
9         ...

```

Listing 34: quantize_model_int8 with exclusions

Embedding and **lm_head** layers are excluded by default because they are particularly sensitive to quantization. Embeddings map discrete token IDs to continuous vectors—small quantization errors can cause tokens to map to entirely different regions of embedding space. The `lm_head` (output projection to vocabulary) directly affects the logits used for sampling; quantization errors here propagate to wrong next-token predictions.

5.2 Simulated INT4 Quantization

```

1 def quantize_per_channel_int4(weight):
2     qweight, scale = quantize_per_channel_int8(weight)
3     qweight_int4 = (qweight + 8).clamp(0, 15).to(torch.uint8)
4     packed = (qweight_int4[::2] & 0x0F) | \
5              ((qweight_int4[1::2] & 0x0F) << 4)
6     return packed, scale

```

Listing 35: INT4 packing (simulated)

4-bit quantization packs two 4-bit values into each byte. The implementation is **simulated**—it quantizes to INT8 first, then remaps to 4-bit range and packs. The benefit is 4× memory reduction vs FP32, 2× vs INT8.

5.3 model_memory_report

Reports estimated memory for FP16, INT8, and INT4 formats. For a 604M parameter model: FP16 $\approx$ 1,137 MB raw (parameter count $\times$ 2 bytes; overhead pushes this higher), INT8 estimated

≈ 570 MB, INT4 estimated ≈ 285 MB.

6 Serving Layer

6.1 Overview

The serving layer wraps the engine in a FastAPI server that implements a subset of the OpenAI completions API:

```

1 GET    /health                $\rightarrow$ {"status": "ok", "gpu": "cuda"}
2 GET    /metrics              $\rightarrow$ {"waiting_requests": N, "
    running_requests": M, ...}
3 POST   /v1/completions       $\rightarrow$ CompletionResponse
4 POST   /v1/chat/completions  $\rightarrow$ ChatCompletionResponse

```

Listing 36: API endpoints

6.2 Request Models

```

1 class CompletionRequest(BaseModel):
2     model: str
3     prompt: str | list[str]
4     max_tokens: int = 64
5     temperature: float = 1.0
6     top_p: float = 1.0
7     top_k: int = 0
8     stop: str | list[str] | None = None
9     stream: bool = False
10    logprobs: int | None = None
11
12 class ChatMessage(BaseModel):
13     role: str
14     content: str
15
16 class ChatCompletionRequest(BaseModel):
17     model: str
18     messages: list[ChatMessage]
19     max_tokens: int = 64
20     temperature: float = 1.0
21     top_p: float = 1.0
22     stream: bool = False

```

Listing 37: Pydantic request models

Chat completions are converted to text completions by formatting messages into a prompt template: each message becomes `<|role|>\n<content>`, and the assistant prefix is appended: `<|assistant|>\n`. This is the Qwen3 chat template format.

6.3 Response Models

```

1 class CompletionResponse(BaseModel):
2     id: str
3     object: str = "text_completion"
4     created: int
5     model: str
6     choices: list[Choice]
7
8 class ChatCompletionResponse(BaseModel):
9     id: str

```

```

10     object: str = "chat.completion"
11     created: int
12     model: str
13     choices: list[ChatChoice]

```

Listing 38: Response models

6.4 Streaming via SSE

When `stream=True`, the `/v1/completions` endpoint returns a `StreamingResponse` with media type `text/event-stream`:

```

1  async def _stream_completions(req, params):
2      request_id = engine.add_request(req.prompt, params)
3      last_len = 0
4      while engine.scheduler.has_unfinished():
5          batch_finished = engine.step()
6          for seq in batch_finished:
7              if seq.request_id == request_id:
8                  gen_text = engine.tokenizer.decode(seq.token_ids[prompt_len
: ], ...)
9                  new_text = gen_text[last_len:]
10                 if new_text:
11                     last_len = len(gen_text)
12                     yield f"data: {json.dumps(chunk)}\n\n"
13                 await asyncio.sleep(0) # yield to event loop
14
15     yield "data: [DONE]\n\n"

```

Listing 39: Streaming response loop

SSE format: Each chunk is `data: <json> \n\n`. The stream ends with `data: [DONE]\n\n`. The `await asyncio.sleep(0)` is critical—it yields control to the asyncio event loop, allowing other concurrent requests (like health checks or other streaming completions) to make progress.

6.5 start_server

```

1  def start_server(engine_instance, SamplingParams, host="0.0.0.0", port=8000):
2      create_app(engine_instance, SamplingParams)
3      uvicorn.run(app, host=host, port=port)

```

Listing 40: Server entry point

`create_app` binds the engine and `SamplingParams` class as module-level globals (a deliberate simplification for educational purposes). In production, you would use dependency injection or a thread-local store.

6.6 metrics Endpoint

```

1  @app.get("/metrics")
2  def metrics():
3      s = engine.scheduler.debug_state()
4      return {
5          "waiting_requests": s["waiting"],
6          "running_requests": s["running"],
7          "free_kv_blocks": s["blocks"]["free_blocks"],
8          "used_kv_blocks": s["blocks"]["used_blocks"],
9          "total_kv_blocks": s["blocks"]["total_blocks"],
10         "engine_steps": engine.step_counter,

```

```
11 }

```

Listing 41: Metrics endpoint

Exposes key scheduler state for monitoring: queue depths, block utilization, and total engine steps. In production, these feed into Prometheus/Grafana dashboards.

7 Multi-Layered Diagnostics

The observability module implements three layers of profiling, each answering a different question:

Layer	Tool	Question
Layer 1	EngineProfiler	<i>What</i> is slow? (schedule/prefill/decode/postprocess)
Layer 2	RequestTracker	<i>How slow</i> for users? (TTFT/TPOT/E2E)
Layer 3	PyTorch Profiler	<i>Which kernel</i> is slow?
Layer 3b	Nsight Systems	<i>Why</i> is the kernel slow? (bandwidth/SM occupancy)

7.1 Layer 1: EngineProfiler

```

1 profiler = EngineProfiler()
2 profiler.start_step(n)
3 profiler.tick("schedule")           # starts measurement
4 # ... scheduler.schedule() ...
5 profiler.tick("schedule")           # records schedule_ms, restarts timer
6 profiler.tick("prefill")            # restarts timer
7 # ... model_runner.run() ...
8 profiler.tick("prefill")            # records prefill_ms
9 profiler.tick("postprocess")         # restarts timer
10 # ... scheduler.postprocess() ...
11 profiler.tick("postprocess")         # records postprocess_ms
12 profiler.end_step(num_prefill=..., num_decode=..., ...)

```

Listing 42: EngineProfiler tick/end_step pattern

Tick semantics: Each `tick(phase)` call records the time elapsed since the *previous* tick as `phase_ms`, then resets the timer. This gives a contiguous waterfall of phase timings with no gaps.

StepMetrics stores per-step breakdown: `schedule_ms`, `prefill_ms`, `decode_ms`, `postprocess_ms`, plus queue/block state.

`print_report()` renders a text-based bar chart showing each phase as a percentage of total step time, and identifies the **bottleneck** phase.

7.2 Layer 2: RequestTracker

Tracks per-request lifecycle events to compute user-facing latency metrics:

```

1 def report(self) -> dict:
2     for rid, r in self.requests.items():
3         if r["first_token_t"]:
4             ttft = (r["first_token_t"] - r["enqueue_t"]) * 1000
5             metrics["ttft_ms"].append(ttft)
6         if r["completion_t"] and r["first_token_t"]:
7             gen_time = (r["completion_t"] - r["first_token_t"]) * 1000
8             tokens = max(r["tokens_generated"], 1)
9             metrics["tpot_ms"].append(gen_time / tokens)
10        if r["completion_t"]:

```

```

11         e2e = (r["completion_t"] - r["enqueue_t"]) * 1000
12         metrics["e2e_ms"].append(e2e)

```

Listing 43: RequestTracker metric computation

Metrics definitions:

- **TTFT** (Time To First Token): $\text{first_token_time} - \text{enqueue_time}$. Measures queue wait + prefill time. Low TTFT is critical for interactive applications (chat, autocomplete).
- **TPOT** (Time Per Output Token): $\frac{\text{completion_time} - \text{first_token_time}}{\text{tokens_generated}}$. Measures decode speed. Also called ITL (Inter-Token Latency). Low TPOT means text streams smoothly.
- **E2E** (End-to-End Latency): $\text{completion_time} - \text{enqueue_time}$. Total wall clock time for a request.

The report includes **p50, p95, p99 percentiles**, not just means. Percentiles capture tail latency—the p99 TTFT of 1.2 seconds means 1% of users wait over a second, even if the mean is 300ms.

7.3 Layer 3: PyTorch Profiler

When the engine profiler says “decode is 62% of step time” but doesn’t explain *why*, the PyTorch Profiler reveals:

```

1 with torch.profiler.profile(
2     activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
3     record_shapes=True,
4 ) as prof:
5     with torch.no_grad():
6         for _ in range(5):
7             _ = model(input_ids)
8
9 print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=12))

```

Listing 44: PyTorch Profiler usage

How to read the output:

- **aten::matmul**: Matrix multiplication kernels. High CUDA time here means the model is compute-bound.
- **aten::scaled_dot_product_attention**: Attention computation. High time here may mean sequence length is large or attention is not using FlashAttention.
- **aten::addmm, aten::bmm**: Batched matrix ops in MLP layers.
- **Self CPU %**: Time spent in the Python/C++ function itself (not its children). High CPU % with low CUDA time means the Python dispatch is the bottleneck.
- **CUDA total**: Total GPU kernel execution time. This is the primary sort key for GPU bottlenecks.

7.4 Layer 3b: NVIDIA Nsight Systems

For deeper GPU analysis, Nsight Systems provides hardware-level profiling:

```

1 nsys profile -o profile_out \
2     python script.py 2>&1
3 nsys-ui profile_out.nsys-rep

```

Listing 45: Nsight Systems command

What to look for in the timeline:

- **Red segments** = CUDA compute kernels (matmul, attention, etc.)
- **Green segments** = GPU-to-GPU memory copies
- **Gaps between kernels** = GPU idle time → CPU/scheduler is the bottleneck
- **SM occupancy**: What percentage of streaming multiprocessors are active?
 - **Low SM occupancy** (< 20%) during decode → memory-bandwidth-bound. Each decode processes only 1 token, so the GPU can't fill its SMs.
 - **High SM occupancy** (> 80%) during prefill → compute-bound. Good—you want SMs saturated when processing many tokens.
- **Many tiny kernel launches** → need CUDA graphs to batch launches.

7.5 diagnose_slowdown()

```

1 def diagnose_slowdown(profiler, tracker):
2     issues = []
3
4     sched_pct = avg_schedule_ms / max(avg_total_ms, 0.001)
5     if sched_pct > 0.50:
6         issues.append("SCHEDULER: >50% of step time in scheduler...")
7
8     if avg_prefill_ms > 100:
9         issues.append("PREFILL: >100ms avg --- need chunked prefill")
10
11    if ttft_p50 > 500:
12        issues.append("TTFT: p50=...ms --- requests waiting too long")
13
14    if tpot_p50 > 50:
15        issues.append("TPOT: p50=...ms --- decode slow, consider quantization")
16
17    return issues or ["No obvious bottleneck detected..."]

```

Listing 46: Automatic bottleneck detection

This function implements **heuristic bottleneck detection** by applying thresholds to key metrics from both Layer 1 and Layer 2:

- **Schedule > 50%**: The scheduler logic itself is too slow—check block allocation algorithms, queue iteration patterns.
- **Prefill > 100ms**: Prompts are too long for single-step prefill. Chunked prefill is the solution.
- **TTFT p50 > 500ms**: Requests are queued too long before starting. Check block availability and admission rate.
- **TPOT p50 > 50ms**: Decode is slow. Investigate model size, quantization, or speculative decoding.

7.6 How the Layers Stack

Layer	Tells You	Does Not Tell You
1 (EngineProfiler)	“Decode is 62% of step time”	<i>Why</i> decode is slow
2 (RequestTracker)	“p95 TTFT is 800ms”	<i>Which</i> request is slow
3 (PyTorch Prof.)	“aten::matmul is 80% of CUDA time”	Whether bandwidth or compute bounds it
3b (Nsight)	“SM occupancy is 12%”	How to fix the model architecture

The diagnostic workflow: Layer 1 identifies the **phase**, Layer 2 identifies the **user impact**, Layer 3 identifies the **kernel**, and Layer 3b identifies the **hardware limitation**. You then fix the root cause (chunk prefill, quantize, use FlashAttention, etc.) and re-run Layers 1–3 to verify improvement.

8 Benchmarking Methodology

8.1 Design Philosophy

Each benchmark is a self-contained Python script that:

1. Accepts a model path as a command-line argument (defaults to `facebook/opt-125m`).
2. Runs multiple configurations (the “independent variable”).
3. Measures elapsed time and generated tokens (the “dependent variable”).
4. Reports throughput in tokens/second.
5. Changes **exactly one variable** between runs.

The benchmarks use small workloads (16 tokens, 12 tokens) to keep runtime under 1 second, making them suitable for quick iteration during development.

8.2 `bench_baseline.py`

Method	Description	Complexity
TinyLLM	Naive: recomputes full context each step	$O(n^2)$
KVLLM	Prefill once, decode with <code>past_key_values</code>	$O(n)$
LLMEngine	Full engine: scheduler, blocks, sampler	$O(n)$ + overhead

Workload: 3 identical prompts of ~ 5 tokens each, generating 16 tokens each. The naive implementation passes the entire token sequence to the model at every step, recomputing attention for all tokens—this is the $O(n^2)$ problem.

8.3 `bench_chunked_prefill.py`

Workload: 1 long prompt (129 words, ~ 200 tokens) + 5 short prompts (~ 5 tokens), 12 tokens each.

Independent variable: `enable_chunked_prefill = True/False`.

Why this workload: The long prompt would otherwise block the 5 short prompts completely until its prefill finishes. Chunked prefill allows the short prompts to interleave, reducing their TTFT.

8.4 `bench_spec_decode.py`

Workload: A prompt with repetitive structure:

“The capital of France is Paris. The capital of Germany is Berlin. The capital of Italy is Rome. The capital of Spain is Madrid. The capital of France is”

Why this works: The last few tokens (“The capital of France is”) appear earlier in the prompt with the continuation “Paris”. The n-gram proposer matches this suffix and proposes “Paris”. **Without repetitive structure, n-gram speculation degenerates to standard decode.**

The benchmark uses a minimal `SpecBenchRunner` that loads the model directly and instantiates a `ModelRunner` with dummy model config (no engine overhead).

8.5 bench_quantization.py

Method: Loads the model in FP16, runs inference, calls `quantize_model_int8()` *in-place* (replacing Linear layers), runs inference again on the same model object, and compares output text and memory reports.

Key check: The INT8 output text should match the FP16 output text. If quantization introduces errors, the generated tokens will diverge.

8.6 bench_profiling.py

Demonstrates all three profiling layers in sequence:

1. **Layer 1+2:** Creates LLM with profiling enabled, runs 3 prompts, prints timeline (first 5 steps + last step), prints engine report and request latency report, runs `diagnose_slowdown()`.
2. **Layer 3:** Runs PyTorch Profiler on 5 forward passes of the raw model, prints kernel table.
3. **Layer 3b:** Prints Nsight Systems usage instructions as an ASCII art box.

9 Benchmark Results

All results measured on Qwen/Qwen3-0.6B with BF16 precision on an RTX 5050 8GB.

9.1 Baseline Comparison

Method	Time (s)	tok/s	Ratio
Naive (no KV cache)	0.954	50.4	1.0×
KV Cache (prefill/decode)	0.601	79.8	1.6×
NanoVLLM++ Engine	0.707	67.9	1.3×

Analysis: The KV cache gives a 1.6× speedup over naive inference. The engine adds scheduler and block manager overhead (encoding, queue management, postprocessing), bringing the speedup down to 1.3× for single-sequence workloads. The engine’s advantage grows with batching—when multiple sequences run concurrently, the scheduler’s overhead amortizes while the KV cache benefit compounds.

9.2 Chunked Prefill

Method	Time (s)	Steps	tok/s
No chunking	1.389	18	51.8
Chunked (512-token chunks)	1.019	18	70.6

1.36× **speedup** with chunked prefill. Although the step count is identical (18 steps), chunked prefill distributes the long prompt’s prefill work across multiple steps, allowing short-prompt decodes to interleave and complete sooner.

9.3 Speculative Decoding (OPT-125M)

Method	Time (s)	Fwds	Accept Rate	Empty Props
Standard	0.232	12	–	–
Spec Decode (n-gram)	0.148	12	0.75	3/12

36% faster with speculative decoding. 9 out of 12 proposed tokens were accepted. 3 proposals were empty (no n-gram match found). The acceptance rate of 0.75 means the target model accepted 3 out of every 4 proposed tokens.

9.4 Quantization (OPT-125M)

Metric	FP16	INT8
Weight memory	238.9 MB	38.4 MB (quantized)
Inference time	0.291s	0.044s
Output correctness	“Paris”	“Paris” (identical)
Layers quantized	–	48 skipped: 72 (embed + lm_head)

The apparent speedup in the INT8 run includes warm-up cache effects and should not be taken as a direct speed comparison. The key result is output correctness: both FP16 and INT8 produce identical generated text.

9.5 Profiling (Qwen3-0.6B)

Metric	Value
TTFT (p50)	386 ms
TPOT (p50)	41 ms
E2E (p50)	812 ms
Decode % of step	62.9%
Bottleneck phase	“decode”
Schedule % of step	8.2%

The engine profiler identifies decode as the bottleneck (62.9% of step time), which is expected—decoding is memory-bandwidth-bound on consumer GPUs. The scheduler overhead is minimal (8.2%), confirming that the scheduling logic is not a bottleneck.

10 GPU Memory Analysis

10.1 Model Weight Memory

For Qwen3-0.6B (604M parameters, BF16):

$$\text{Model memory} = 604 \times 10^6 \times 2 \text{ bytes} \approx 1.21 \text{ GB}$$

This includes all linear layer weights, embedding tables, layer norms, and biases. The embedding table (**wte**: $152064 \times 1024 \approx 156\text{M}$ elements $\approx 312\text{MB}$) and **lm_head** are the largest individual tensors.

10.2 KV Cache Memory Per Block

$$\begin{aligned}\text{KV per token} &= 2 \times L \times H_{kv} \times d_h \times 2 \text{ bytes} \\ &= 2 \times 28 \times 8 \times 128 \times 2 \\ &= 114,688 \text{ bytes} \approx 112 \text{ KB}\end{aligned}$$

$$\text{KV per block (256 tokens)} = 112 \text{ KB} \times 256 \approx 28.7 \text{ MB}$$

10.3 Total KV Blocks Estimation

On the RTX 5050 with 8 GB total VRAM, using `gpu_memory_utilization = 0.90`:

$$\begin{aligned}\text{Available total} &= 8 \times 10^9 \times 0.90 \approx 7.2 \text{ GB} \\ \text{Model memory} &\approx 1.21 \text{ GB} \\ \text{Runtime overhead} &\approx 0.5 \text{ GB} \\ \text{Available for KV} &= 7.2 - 1.21 - 0.5 \approx 5.49 \text{ GB} \\ \text{Num KV blocks} &= \left\lfloor \frac{5.49 \times 10^9}{28.7 \times 10^6} \right\rfloor \approx 191 \text{ blocks}\end{aligned}$$

191 blocks at 256 tokens per block = 48,896 token positions across all concurrent sequences. At `max_model_len = 8192`, this supports up to 6 fully-utilized sequences. With shorter prompts (typical chatbot conversations), it supports far more.

10.4 Why `gpu_memory_utilization = 0.90`

The 10% headroom is critical. Without it:

- PyTorch’s CUDA caching allocator needs scratch space for temporary tensors (activations, gradients during warmup).
- CUDA context initialization consumes ~200–500 MB.
- BF16 model loading may allocate intermediate FP32 buffers before casting.
- NVIDIA driver and other processes (X server, compositor) consume GPU memory.

Setting `gpu_memory_utilization` too high (> 0.95) causes CUDA OOM errors during peak load. Setting it too low wastes KV cache capacity. 0.90 is the standard vLLM default and a well-tested balance.

11 Debugging Section

11.1 OPT vs Qwen3 Config Key Names

Bug: `AttributeError` when loading Qwen3 after developing with OPT.

Root cause: Different HuggingFace models use different attribute names for the same semantic information:

- `intermediate_size` (Qwen3, Llama, Mistral) vs `ffn_dim` (OPT)
- `num_key_value_heads` (Qwen3, for GQA) vs absent (OPT, no GQA, falls back to `num_attention_heads`)
- `head_dim` (Qwen3) vs absent (OPT, computed as `hidden_size / num_attention_heads`)

Fix: Use `getattr(hf_config, primary_key, getattr(hf_config, fallback_key, 0))` for all config reads.

11.2 Initialization Order

Bug: `AttributeError` in `_estimate_kv_blocks()` because `self.model` didn't exist yet.

Root cause: `_estimate_kv_blocks()` iterates `self.model.parameters()` to compute weight memory, but the model was loaded *after* KV block estimation.

Fix: Reorder initialization: load model $\rightarrow$ build `model_config` $\rightarrow$ estimate KV blocks $\rightarrow$ create `BlockManager` $\rightarrow$ create `ModelRunner` $\rightarrow$ create `Scheduler`. This is now the canonical order in `LLMEngine.__init__`.

11.3 KV Cache OOM on Qwen3

Bug: Pre-allocating 191 blocks of KV cache as full tensors ($28 \times 8 \times 128$ float16 per block, times 191 blocks, times 2 for K+V) consumed ~ 2.57 GB. Combined with model weights (1.21 GB) and runtime overhead, this exceeded the 8 GB VRAM budget, causing CUDA OOM.

Fix: Replaced the pre-allocated paged KV cache with HuggingFace's `past_key_values` mechanism. Each sequence stores its own HF `DynamicCache` object, and HF internally manages the tensor allocations. This is functionally equivalent for educational purposes and avoids manual VRAM management.

11.4 Gibberish Output Without KV Cache

Bug: The model produced random/gibberish output during decode. Each decode step generated nonsensical tokens with no relationship to the prompt.

Root cause: When `past_key_values` is `None`, `run_decode` calls `self.model(input_ids, use_cache=False)`. This feeds **only the last token** (a single token ID) to the model. Without the KV cache, the model has **zero context**—it sees one token with no preceding history and predicts the next token based solely on that single input. The result is effectively random.

Fix: Always pass `past_key_values` when it exists. The KV cache is what gives the model access to all previous tokens without recomputing them.

11.5 Profiler Tick Placement

Bug: The engine profiler showed model forward time under “sample” instead of “prefill” or “decode”.

Root cause: The profiler ticks were placed incorrectly—the measurement window for the model forward was not properly bounded by prefill/decode ticks.

Fix: Wrap `model_runner.run()` between `tick(model_kind)` calls, where `model_kind` is dynamically determined as “prefill” or “decode” based on the batch type.

11.6 RequestTracker TPOT Calculation

Bug: TPOT (time per output token) showed impossibly large values (e.g., 500ms per token) when the actual decode was ~ 40 ms/token.

Root cause: `tokens_generated` was recorded as 0 in `record_completion()`, causing division by `max(tokens, 1) = 1` and inflating TPOT by the number of tokens.

Fix: Changed `record_completion()` to accept `num_tokens` as a parameter, and pass `seq.num_generated_tokens` from the engine's postprocessing step.

12 Interview Preparation — Q&A

12.1 Core Inference Concepts

Q: Why is naive autoregressive generation $O(n^2)$?

A: Each step processes the entire sequence of length n , so step k processes k tokens. Total tokens processed for N generated tokens: $P + (P + 1) + (P + 2) + \dots + (P + N) \approx N \cdot P + \frac{N^2}{2}$, where P is prompt length. The N^2 term dominates for long generations.

Q: What does the KV cache store?

A: During prefill, each transformer layer computes Key (K) and Value (V) projections for every token in the prompt. These are stored and reused during decode: when generating token $t + 1$, the model only needs to compute Q_{t+1} (query for the new token), then attend over all stored K and V from tokens 0 through t . Without KV cache, K and V for all previous tokens would be recomputed each step.

Q: Why is prefill compute-bound but decode memory-bound?

A: Prefill processes P tokens in parallel—each layer runs a $P \times d_{\text{model}}$ matrix multiply, saturating all GPU streaming multiprocessors (SMs). Decode processes 1 token per step—the matrix multiply is $1 \times d_{\text{model}}$, which under-utilizes SMs. The limiting factor becomes how fast weights can be read from GPU memory (HBM bandwidth), not how fast they can be computed. Modern GPUs have $\sim 1\text{--}2$ TB/s memory bandwidth and $\sim 100\text{--}300$ TFLOPS compute; for a 604M parameter model, the arithmetic intensity of decode is too low to saturate compute.

Q: Why paged KV cache?

A: Without paging, KV cache memory must be pre-allocated contiguously for the maximum sequence length (e.g., 8192 tokens). For 100 concurrent sequences, this wastes $100 \times 8192 \times \text{kv_per_token}$ bytes, even if most sequences are short. Paging allocates memory in blocks (e.g., 256 tokens each), and sequences only consume blocks they actually need. This is analogous to virtual memory: the block table maps logical blocks (the sequence’s view) to physical blocks (actual GPU memory).

Q: What does reference counting prevent?

A: Use-after-free. When two sequences share a prefix block (prefix caching), deallocating the block when the first sequence finishes would corrupt the second sequence’s KV cache. Reference counting ensures the block is freed only when **all** sequences using it have finished.

Q: How does chained hash prefix caching work?

A: Block i ’s hash = `hash(block_i - 1_hash, tokens_in_block_i)`. If two sequences share the first 512 tokens, they each have identical block-0 hashes and identical block-1 hashes. The BlockManager detects this via `hash_to_block_id` and reuses the physical blocks. The “chained” aspect means blocks can’t be matched in isolation—the entire prefix must match.

Q: What is continuous batching?

A: Instead of processing one request to completion before starting the next (static batching), continuous batching adds new requests to the running batch as soon as they arrive, and removes finished requests immediately. A decode batch might contain sequences at different generation steps—some on step 3, some on step 47. The scheduler handles this by scheduling a 1-token decode for each running sequence per step.

Q: How does chunked prefill reduce TTFT?

A: Without chunking, a 4096-token prompt must be fully prefilled before any decode can happen.

With chunking, the prompt is split into (say) eight 512-token chunks. After chunk 1 prefills, the scheduler can run decode steps for other sequences. This means short-prompt requests start generating tokens while the long prompt is still being prefilled, dramatically reducing their TTFT.

12.2 Speculative Decoding

Q: Why does speculative decoding reduce latency?

A: The key insight is that one forward pass with k tokens costs **roughly the same** as one forward pass with 1 token during decode, because both are memory-bandwidth-bound (weight reads dominate). By proposing k candidate tokens and verifying them in one forward pass, you generate up to k tokens for the cost of ~ 1 . With acceptance rate α , the effective speedup is $\frac{k}{1 + \frac{k}{\alpha}}$ for draft-model-based speculation. With n-gram (no draft model cost), the speedup approaches $\alpha \cdot k$.

Q: What is a proposer/verifier?

A: The **proposer** generates a sequence of candidate tokens. The **verifier** checks these candidates against the target model: for each candidate, it compares the target model's prediction. If they match, the candidate is accepted. If they diverge, the target model's token is used and remaining candidates are discarded. This guarantees that the output is **identical** to the target model (for greedy verification) or **distributionally identical** (for rejection sampling).

Q: Why does acceptance rate matter?

A: If acceptance rate is 0.25, speculating 4 tokens means 3 are rejected on average. The verification forward processes 4 tokens but only keeps ~ 1.25 (1 accepted + 0.25 average position before rejection). The net speedup is minimal or negative (proposal + verification overhead > savings). Acceptance rate must be high enough (> 0.5) for speculation to be worthwhile.

Q: Why is spec decode not always faster at high QPS?

A: At high QPS, the GPU processes large decode batches (e.g., 128 sequences $\times$ 1 token = 128 tokens per forward). This increases arithmetic intensity, making decode compute-bound instead of memory-bound. The speculative overhead (proposal computation, verification) competes for the same compute resources, potentially reducing throughput.

12.3 Quantization

Q: Weight-only vs KV cache quantization?

A: **Weight-only** quantization reduces model parameter memory. **KV cache quantization** reduces the memory used by stored keys and values. Weight-only helps fit larger models in the same VRAM; KV cache quantization helps support longer sequences or more concurrent requests. The two are complementary and often used together.

Q: Per-channel vs per-tensor?

A: Per-tensor uses one scale for the entire matrix—fine if all rows have similar magnitudes, but LLM weight matrices often have $1000\times$ variation between rows. Per-channel gives each row its own scale, using the full quantization range for every row and preserving more accuracy at the cost of storing N_{rows} scale values instead of 1.

Q: Why do packed kernels matter?

A: Naive dequantization (load INT8 $\rightarrow$ convert to FP16 $\rightarrow$ multiply) adds conversion overhead that can negate memory savings. **Fused dequantize + matmul kernels** (e.g., AWQ, GPTQ,

Marlin) read INT8/INT4 weights directly and perform multiply-accumulate in higher precision, achieving both memory savings *and* speed improvements by reducing memory bandwidth.

12.4 Serving

Q: What changes from offline to online inference?

A: Offline inference (`llm.generate(prompts)`) runs all prompts to completion and returns results. Online serving adds: (1) **concurrent requests** arriving at different times—the scheduler must admit them mid-batch; (2) **streaming**—tokens are sent to the client as they’re generated, not all at once; (3) **cancellation**—clients may disconnect mid-generation; (4) **metrics/monitoring**—operators need visibility into queue depths, latency, and error rates.

Q: How does streaming work?

A: Each generated token is sent as an SSE event: `data: {"choices": [{"text": "tok"}]}\n\n`. The client reads a continuous stream and updates the UI incrementally. This gives the perception of low latency even if the full generation takes seconds.

Q: How to expose metrics?

A: The `/metrics` endpoint exposes scheduler state (waiting/running queues, block utilization, step count). In production, this feeds into Prometheus which scrapes the endpoint periodically, and Grafana visualizes dashboards with queue depth over time, latency percentiles, and block pressure.

12.5 Diagnostics

Q: How do you find the bottleneck in an inference engine?

A: Three-layer approach:

1. **Engine profiler**: Measures time spent in each phase (schedule, prefill, decode, postprocess). If decode is 80%, focus on model performance.
2. **Request tracker**: Measures TTFT and TPOT. If TTFT is high but TPOT is low, the problem is queue wait or prefill—not decode.
3. **GPU profiler**: If decode is confirmed slow, profile with PyTorch Profiler to find which kernel (matmul, attention, layer norm) is the hotspot. Then Nsight to determine if it’s bandwidth-bound or compute-bound.

Q: When to use Nsight vs PyTorch Profiler vs custom instrumentation?

A:

- **Custom (EngineProfiler)**: Always. It’s low-overhead and identifies *which logical component* is slow. First line of defense.
- **PyTorch Profiler**: When custom profiling says “model forward is slow” and you need to know *which kernel*.
- **Nsight Systems**: When PyTorch Profiler shows a kernel is slow but you don’t know *why*—is it bandwidth, compute, or launch overhead? Also for CUDA graph optimization and stream analysis.

Q: What does SM occupancy tell you?

A: SM (Streaming Multiprocessor) occupancy measures what percentage of the GPU’s compute units are active. Low occupancy during decode ($< 20\%$) indicates memory-bandwidth limitation—the GPU can’t keep its SMs fed because it’s waiting on memory reads. High occupancy during prefill ($> 80\%$) is good—the GPU is efficiently processing many tokens in parallel.

If prefill shows low occupancy, the batch size may be too small or kernels too fine-grained.

13 Glossary

Term	Definition
Prefill	The first forward pass where the full prompt is processed. All prompt tokens are fed simultaneously, and Key/Value projections are computed and cached. Prefill is compute-bound because P tokens are processed in parallel.
Decode	Each subsequent forward pass where exactly one new token is generated autoregressively. The model computes the Query for the new token and attends over all cached Keys/Values. Decode is memory-bandwidth-bound.
KV Cache	Storage for the Key and Value projections computed during prefill. Avoids recomputing attention for all previous tokens at each decode step. The fundamental optimization that makes autoregressive generation practical.
PagedAttention	A memory management technique where KV cache memory is divided into fixed-size blocks (pages). Each sequence maintains a block table mapping logical block indices to physical block IDs. Enables memory sharing (prefix caching), dynamic allocation, and eliminates internal fragmentation.
Block Table	A per-sequence mapping from logical block indices to physical block IDs. This is the indirection that makes PagedAttention work—the model accesses KV cache through the block table, unaware of physical memory layout.
Continuous Batching	A scheduling policy where new requests can be added to a running batch at any engine step, and finished requests are removed immediately. Contrasts with static batching (fixed batch, process to completion, then next batch).
Chunked Prefill	Splitting a long prompt’s prefill into smaller chunks (e.g., 512 tokens each) and interleaving prefill chunks with decode steps. Reduces TTFT for waiting requests and prevents decode starvation by long prompts.
Prefix Caching	Reusing KV cache blocks across sequences that share the same prompt prefix. Implemented via chained hashing: if two sequences share the first N tokens, they share the first $\lceil N/\text{block_size} \rceil$ physical blocks.
TTFT	Time To First Token. The wall-clock time from when a request is enqueued to when the first generated token is produced. Measures queue wait + prefill time. Critical for interactive applications.
TPOT	Time Per Output Token. The average time between generated tokens during decode. $\text{TPOT} = (\text{completion_time} - \text{first_token_time}) / \text{tokens_generated}$. Also called ITL (Inter-Token Latency).
ITL	Inter-Token Latency. Synonymous with TPOT. The time between consecutive token generations during decode. Low ITL means text streams appear smooth and responsive.
Speculative Decoding	A technique where a fast “proposer” generates candidate tokens, and the target model verifies them in one forward pass. If k tokens are proposed and $\alpha \cdot k$ are accepted, effective throughput increases by $\alpha \cdot k\times$.

Term	Definition
LoRA	Low-Rank Adaptation. A parameter-efficient fine-tuning method that adds trainable low-rank matrices ($A \cdot B$) to frozen pre-trained weights. Enables serving multiple fine-tuned variants of the same base model without duplicating all parameters.
Guided Decoding	Constraining the model’s output to follow a specific format (JSON schema, regex, grammar). The sampler limits its token selection to only those that maintain valid structure according to the provided grammar.
P/D Disaggregation	Prefill/Decode disaggregation. Running prefill and decode on separate GPU nodes, each optimized for its workload: prefill nodes need high compute throughput; decode nodes need high memory bandwidth. Enables independent scaling.
GQA	Grouped Query Attention. A variant of multi-head attention where multiple query heads share the same key-value head. In Qwen3-0.6B, 16 query heads share 8 KV heads (GQA ratio 2:1). Reduces KV cache size by factor of H/H_{kv} while maintaining quality. More generally: MQA (Multi-Query Attention, all queries share one KV) and MHA (Multi-Head Attention, one KV per query).
SM Occupancy	Streaming Multiprocessor occupancy. The percentage of a GPU’s compute units that are actively processing work. High occupancy ($> 80\%$) means good utilization; low occupancy during decode ($< 20\%$) indicates memory-bandwidth bottleneck.
Token Budget	The maximum number of tokens that can be processed in a single engine step. Key fairness mechanism: prevents long prefill sequences from starving decode sequences.
Reference Counting	A memory management technique where each resource (KV block) tracks how many sequences are using it. The resource is freed only when the count drops to zero. Prevents use-after-free with shared prefix blocks.
Per-Channel Quantization	Quantization where each row (output channel) of a weight matrix gets its own scale factor. Preserves more accuracy than per-tensor quantization because different output channels have different magnitude ranges.
CUDA Graph	A pre-recorded sequence of CUDA kernel launches that can be replayed with a single API call. Eliminates per-kernel launch overhead, which is significant for decode steps (many tiny kernel launches).
SSE	Server-Sent Events. A unidirectional streaming protocol where the server pushes data to the client. Used for token-by-token streaming in OpenAI-compatible APIs. Each event is <code>data: <json>\n\n</code> .
FlashAttention	An IO-aware exact attention algorithm that tiles the attention computation to minimize HBM reads/writes. Uses online softmax tiling and recomputation instead of materializing the full $N \times N$ attention matrix. Standard in modern LLM inference.

A Codebase Lineage

NanoVLLM++ draws architectural inspiration from the following projects:

- **vLLM** (Kwon et al., 2023): PagedAttention, continuous batching, token-budget scheduling.
- **SGLang** (Zheng et al., 2024): RadixAttention prefix caching, chunked prefill scheduling.
- **DeepSpeed-MII**: Model parallelism and quantization integration patterns.
- **TensorRT-LLM**: CUDA graph and kernel fusion patterns.

The educational implementation simplifies every component: no CUDA kernels, no model parallelism, no KV eviction, no async scheduling. The goal is conceptual understanding, not production throughput.

B Key Design Decisions

1. **Per-sequence model forward (not batched)**: The current `ModelRunner.run()` iterates sequences one at a time rather than batching them into a single forward pass. This is the primary simplification vs production vLLM, which uses CUDA graphs and dynamic batching. A true batched prefill/decode would require packing variable-length sequences, constructing batched attention masks, and managing ragged KV caches.
2. **HF past_key_values instead of manual KV pages**: The `BlockManager` tracks logical-to-physical mappings but doesn't actually store KV tensors. The HuggingFace `DynamicCache` handles KV memory internally. This was necessary for the 8 GB VRAM constraint.
3. **Synchronous step loop**: The engine iterates `step()` synchronously in a `while` loop. Production engines run an async event loop where steps are triggered by an internal clock or request arrival events, enabling true concurrent request handling.
4. **No CUDA graphs**: `enforce_eager` defaults to `False` but CUDA graphs are not implemented. They would require capturing model forward with fixed input shapes, replaying with minimal CPU involvement.

C Future Work

- **LoRA adapters**: Hot-swappable low-rank adapter weights for multi-tenant serving.
- **KV eviction simulator**: Simulate and benchmark eviction policies (LRU, LFU, hybrid) when blocks are scarce.
- **Disaggregated prefill simulator**: Separate prefill and decode instances communicating via NCCL/RDMA.
- **RL rollout mode**: Generate multiple completions per prompt for reinforcement learning training loops.
- **Batched model forward**: Pack multiple sequences into a single forward pass with proper attention masking.